TP — Sécuriser le backend d'un gestionnaire de tâches

Spring Boot, Spring Security et client React Planora

Durée : **60 minutes**, après une présentation théorique de 30 minutes. Travail individuel ou en binôme. Technologies : Java 17, Maven, Spring Boot et React/Vite.

Nom et prénom : ……
Groupe : …… **Date :** ……

1. Le produit et le travail demandé

Planora est une application de gestion des tâches d'une équipe. Son interface React et son API métier sont déjà fournies : on peut consulter, créer, modifier, terminer et supprimer des tâches. Les statistiques proviennent des tâches réellement stockées par Spring.

Le TP fait évoluer **le backend Spring**, en ajoutant progressivement l'authentification puis les autorisations. Utilisez l'application pour constater les effets de vos changements. Les règles d'accès doivent être imposées par Spring ; aucune modification de React n'est demandée.

À la fin, vous saurez distinguer authentification et autorisation, déclarer des comptes avec BCrypt, configurer `SecurityFilterChain`, protéger une méthode avec `@PreAuthorize` et vérifier les parcours métier avec les tests et les réponses HTTP.

Les tâches sont partagées par l'équipe : **Alice et Bob peuvent gérer toutes les tâches**. Le TP ne met pas en place de propriété individuelle des tâches. Les statistiques et le projet confidentiel sont réservés à Bob, administrateur.

Contrat de l'API

Méthode et route	Fonction	Succès
`GET /api/public/info`	Information publique, à consulter avec curl	`200`, texte
`GET /api/tasks`	Liste des tâches	`200`, tableau JSON
`POST /api/tasks`	Création	`201`, tâche JSON
`PUT /api/tasks/{id}`	Modification	`200`, tâche JSON
`PATCH /api/tasks/{id}/complete`	Passage à l'état terminé	`200`, tâche JSON
`DELETE /api/tasks/{id}`	Suppression	`204`, sans corps
`GET /api/admin/stats`	Statistiques calculées	`200`, objet JSON
`GET /api/tasks/confidential`	Note du projet Orion, fournie à partir de la branche 5	`200`, objet JSON

Le CRUD et les statistiques existent dès la branche 1. La route confidentielle est absente des branches 1 à 4 ; elle est déjà implémentée dans la branche 5, où il reste à la protéger.

Une tâche contient `id`, `title`, `description`, `status`, `priority`, `dueDate` et `assignee`. Le formulaire React permet de modifier le titre, la description, l'état, la priorité et l'échéance. La personne assignée est affichée ; elle n'est pas éditable dans ce formulaire et le serveur la conserve lors d'une modification.

États : `TODO`, `IN_PROGRESS`, `DONE`. Priorités : `LOW`, `MEDIUM`, `HIGH`. Le titre est obligatoire et limité à 160 caractères ; la description à 1 000 caractères ; l'échéance utilise le format `AAAA-MM-JJ`. Des données invalides renvoient `400` et un identifiant de tâche inexistant renvoie `404`, si les filtres de sécurité laissent passer la requête.

Les données restent **en mémoire**. Un redémarrage de Spring restaure les trois tâches d'exemple. Au démarrage, les statistiques sont : 3 tâches, 1 terminée, 2 en attente, 1 de priorité haute et 33 % d'achèvement. Les statistiques d'administration sont calculées sur toutes les tâches ; les compteurs de l'écran des tâches utilisent la liste chargée dans React.

Architecture

Partie	Responsabilité
`frontend/src/main.jsx`	Écrans métier et formulaire de connexion
`frontend/src/api.js`	Appels JSON et en-tête HTTP Basic

Partie	Responsabilité
`controller/`	Routes HTTP et appels des services
`model/`	Objets échangés avec React
`service/TaskService.java`	CRUD et stockage en mémoire
`service/StatisticsService.java`	Indicateurs calculés
`config/UserConfig.java`	Comptes, rôles et encodeur BCrypt
`config/SecurityConfig.java`	Règles HTTP et sécurité des méthodes

Les chemins Java sont relatifs à `src/main/java/fr/tpspringsecurity/`. Les fichiers de configuration apparaissent au fil des branches. La même interface React est présente sur les six branches.

2. Comprendre les branches

Chaque branche est **le point de départ de son exercice**, avec les corrections précédentes. La branche 6 contient la solution finale. Les numéros ne désignent donc pas des exercices déjà terminés.

Branche	État initial	Travail demandé	Durée
`Tp_version1`	CRUD et statistiques ouverts	Auditer les accès	10 min
`Tp_version2`	API encore ouverte, dépendance à ajouter	Installer Spring Security	10 min
`Tp_version3`	Sécurité par défaut, compte généré	Déclarer Alice et Bob	15 min
`Tp_version4`	Alice et Bob présents, politique par défaut	Configurer les droits HTTP et CSRF	10 min
`Tp_version5`	Droits HTTP corrigés ; note lisible par tout compte valide	Activer la protection de méthode	10 min
`Tp_version6`	Sécurité complète	Vérifier le parcours final	5 min

Après l'exercice 2 et après l'exercice 3, les lectures demandent un compte valide, y compris l'information publique. Les rôles ne limitent pas encore les statistiques. **Sans jeton CSRF, les mutations renvoient `403`, même avec un compte valide et même sans compte.** L'exercice 4 adapte cette configuration pour permettre les mutations du client fourni.

Le compte généré `user` apparaît après l'ajout de la dépendance en exercice 2 et reste présent au début de la branche 3. Alice et Bob existent **après la réalisation de l'exercice 3**, et dès l'ouverture des branches 4 à 6.

3. Préparer et lancer le projet

Prérequis : JDK 17, Maven 3.6.3 ou supérieur, Git et Node.js **20.19+ ou 22.12+** avec npm. Vérifiez notamment que `mvn -version` utilise le JDK attendu.

```
java -version
mvn -version
node --version
npm --version
```

Récupérez le dépôt :

```
git clone https://github.com/romarich57/Tp_Spring_security.git
cd Tp_Spring_security
git switch Tp_version1
```

Dans un premier terminal, à la racine :

```
mvn test
mvn spring-boot:run
```

Dans un second terminal :

```
cd frontend
npm install
npm run dev
```

Ouvrez **http ://localhost :5173**. L'API écoute sur **http ://localhost :8080**. Vite relaie `/api` vers Spring, ce qui permet à React de travailler sur la même origine via le proxy.

Utiliser Planora

- **Mes tâches** : consulter, rechercher, filtrer, créer, modifier, terminer et supprimer.
- **Statistiques** : consulter les indicateurs du backend.
- **Projet confidentiel** : lire la note lorsque le backend propose cette route et autorise l'accès.
- **Connexion** : saisir le nom du compte et son mot de passe ; se déconnecter pour revenir au parcours anonyme.

Le formulaire transmet les identifiants en HTTP Basic à chaque requête. Ils restent en mémoire React, sans cookie ni stockage local ; recharger la page les efface. Il n'existe pas de route de login dédiée. L'affichage du nom saisi ne prouve pas que le compte est valide : Spring vérifie les identifiants à chaque appel et peut renvoyer `401`.

L'interface affiche les données et les erreurs réelles du serveur. Pour relever précisément un statut de succès, utilisez l'onglet Réseau du navigateur, `requests.http` ou `curl`. L'information publique n'a pas d'écran dans Planora : vérifiez-la avec `curl`.

Passer à la branche suivante

Arrêtez Spring et Vite avec `Ctrl+C`. Enregistrez les fichiers de votre exercice dans un commit local, puis changez de branche et relancez les deux serveurs. Exemple après l'exercice 2 :

```
git add pom.xml src/test/java/fr/tpspringsecurity/TaskApiSecurityTest.java
git commit -m "Exercice 2 terminé"
git switch Tp_version3
```

Les branches suivantes fournissent déjà les corrections précédentes ; il n'est pas nécessaire de reporter votre commit sur elles. Vous pouvez aussi mettre vos changements de côté avec `git stash push -m "Mon exercice"` ; ne réappliquez pas automatiquement ce stash sur la branche suivante.

4. Exercice 1 — Enquête : contrôle d'accès cassé (10 min)

Branche : `Tp_version1`. Dans cette première étape, ne corrigez pas le code : votre objectif est d'observer, de démontrer et d'expliquer.

Point de départ : OWASP Top 10

L'OWASP Top 10 :2025, A01 — Broken Access Control décrit les situations où une application laisse quelqu'un agir au-delà des permissions prévues. Parmi les exemples cités : une API qui n'applique pas de contrôle aux opérations `POST`, `PUT` ou `DELETE`, ou un accès à une fonction privilégiée en appelant directement son URL. Dans cet exercice, recherchez des indices de cette catégorie dans Planora. Vous devrez établir vous-même quels accès sont problématiques et proposer les règles à appliquer.

Mener l'enquête

1. Lancez Spring Boot et Planora. Restez **déconnecté**.
2. Dans **Mes tâches**, consultez les tâches, puis créez-en une, modifiez-la, terminez-la et supprimez-la. Notez ce que l'interface vous laisse faire.
3. Ouvrez **Statistiques**. La navigation dans React est un indice à relever, pas une preuve suffisante du contrôle d'accès.
4. Vérifiez les requêtes directement contre l'API : observez les appels dans l'onglet **Réseau** du navigateur ou utilisez `curl`. Testez les routes listées dans la fiche ci-dessous **sans identifiants**. Pour les opérations de modification, utilisez un identifiant de tâche existant ; supprimez uniquement la tâche que vous venez de créer.
5. Comparez ce que vous observez avec les permissions que vous estimez nécessaires pour une application d'équipe. Pour vous guider, regardez qui peut lire les tâches, modifier les données de l'équipe et consulter des statistiques d'administration.
6. Lisez `TaskController`, `AdminController`, `TaskService` et `StatisticsService`. Repérez où sont déclarées les routes et où se trouve (ou ne se trouve pas) une vérification d'accès.

L'interface publique `/api/public/info` peut être vérifiée avec `curl` ; elle n'a pas d'écran dédié dans Planora. Les réponses initiales de l'API sont observables ainsi :

```
curl -i http://localhost:8080/api/tasks
curl -i http://localhost:8080/api/admin/stats
curl -i http://localhost:8080/api/public/info
```

Ne vous contentez pas de dire qu'une opération « fonctionne » : gardez le statut HTTP, le résultat visible et l'action réalisée comme éléments de preuve.

Fiche d'observation à compléter

La fiche ci-dessous fait partie de ce support. Complétez-la dans le PDF (ou imprimez ces pages). Pour chaque requête, notez ce que **vous** avez observé sans vous connecter. Si une réponse diffère de ce que vous attendiez, expliquez pourquoi cela vous semble important.

Requête testée sans connexion	Accessible ?	Statut HTTP	Résultat ou effet observé
`GET /api/tasks`			
`POST /api/tasks`			
`PUT /api/tasks/{id}`			
`PATCH /api/tasks/{id}/complete`			
`DELETE /api/tasks/{id}`			
`GET /api/admin/stats`			

1. Quels droits proposeriez-vous ? Indiquez ce qu'une personne sans compte, un membre connecté et un administrateur devraient pouvoir faire dans cette application. Justifiez brièvement.

— Une personne sans compte : _____
— Un membre connecté de l'équipe : _____
— Un administrateur : _____

2. Que vous apprend l'enquête sur OWASP A01 ? Comparez les droits proposés aux résultats observés. Donnez deux exemples précis de requêtes qui vous semblent problématiques et expliquez le risque associé.

_____

_____

_____

3. Où faut-il vérifier les droits ? Expliquez avec vos mots la différence entre authentification et autorisation. Pourquoi le contrôle doit-il être appliqué dans le backend, même si React peut masquer des boutons ?

_____

_____

_____

Conclusion — En une phrase, reliez vos observations à **A01 :2025 Broken Access Control**. Gardez cette fiche pour comparer les résultats après les exercices suivants.

5. Exercice 2 — Activer la protection par défaut (10 min)

Branche : `Tp_version2`. **Fichier à modifier :** `pom.xml` uniquement.

Objectif

Ajouter Spring Security au projet et observer ce que Spring Boot met en place automatiquement, sans écrire encore de règles personnalisées. Vous allez constater que la protection par défaut demande une authentification pour toutes les routes, même celle nommée `/api/public/info`. Spring crée aussi temporairement le compte `user` et affiche son mot de passe dans la console.

À la fin de l'exercice, vous devriez pouvoir expliquer ce que la dépendance change, comment retrouver le compte de développement et pourquoi les rôles (USER/ADMIN) ne sont pas encore distingués.

TODO 2.1 — Ajouter la dépendance

Dans `pom.xml`, repérez le commentaire `TODO EXERCICE 2`. Remplacez-le par ce bloc Maven :

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-security</artifactId>
</dependency>
```

N'ajoutez pas de balise `<version>` : le parent Spring Boot gère cette version. N'ajoutez pas de configuration Java et ne modifiez pas les contrôleurs. À cette étape, il suffit d'ajouter le starter.

TODO 2.2 — Observer l'API sans identifiants

Redémarrez Spring avec `mvn spring-boot:run`. Dans Planora, sans vous connecter, consultez **Mes tâches** puis **Statistiques**. Relevez les réponses dans l'onglet **Réseau** du navigateur. L'information `/api/public/info` n'a pas d'écran dans React ; vérifiez-la directement :

```
curl -i http://localhost:8080/api/public/info
curl -i http://localhost:8080/api/tasks
curl -i http://localhost:8080/api/admin/stats
```

À observer : les requêtes de lecture renvoient `401 Unauthorized`, y compris l'information publique. Expliquez pourquoi un nom de route contenant `public` ne suffit pas à la rendre accessible.

TODO 2.3 — Retrouver l'identité temporaire

Dans la console du backend, repérez le message Spring qui affiche le **mot de passe généré**. Il change au démarrage : utilisez celui de l'exécution en cours et ne le copiez pas dans le dépôt.

Dans Planora, ouvrez **Connexion**, saisissez l'identifiant `user` et ce mot de passe, puis consultez les tâches. Vérifiez aussi la requête avec curl :

```
curl -i -u user:MOT_DE_PASSE_GENERE http://localhost:8080/api/tasks
```

Remplacez `MOT_DE_PASSE_GENERE` par la valeur de votre console. La lecture doit fonctionner après authentification. Le fait que la page React affiche `user` ne suffit pas à prouver la connexion : vérifiez la réponse de l'API.

TODO 2.4 — Comparer une lecture et une mutation

Essayez de créer une tâche depuis Planora avec le compte `user`, puis notez le statut HTTP. Spring Security active aussi la protection CSRF par défaut ; le client pédagogique n'envoie pas de jeton CSRF. Une mutation peut donc recevoir `403` même avec le bon mot de passe. Les lectures `GET`, elles, sont acceptées avec les identifiants valides.

Ne désactivez pas CSRF dans cet exercice : cette décision fait partie de la configuration de l'API à l'exercice 4. Pour l'instant, distinguez le refus d'authentification (`401`) du refus d'une mutation sans jeton CSRF (`403`).

TODO 2.5 — Vérifier avec le test fourni

Dans `src/test/java/fr/tpspringsecurity/TaskApiSecurityTest.java`, repérez `exercice2LesRoutesDemandentUneConn`
Après avoir ajouté la dépendance, retirez l'annotation `@Disabled` juste au-dessus de cette méthode, puis exécutez :

```
mvn test
```

Le test vérifie que les trois lectures (`/api/public/info`, `/api/tasks`, `/api/admin/stats`) refusent un visiteur anonyme. Il ne vérifie pas les mutations.

Questions de compréhension

1. Qui crée le compte `user`, puisque vous ne l'avez pas déclaré dans le code ?
2. À cette étape, quelle règle permet de distinguer un utilisateur d'un administrateur ? Appuyez-vous sur ce que vous avez observé.
3. Pourquoi la route d'information publique exige-t-elle aussi une authentification ?
4. Pourquoi une mutation peut-elle renvoyer `403` alors que votre lecture authentifiée renvoie `200` ?

Pour continuer : la branche `Tp_version3` garde la protection par défaut et remplace le compte temporaire par Alice et Bob.

6. Exercice 3 — Déclarer les comptes avec BCrypt (15 min)

Branche : `Tp_version3`. **Fichier à modifier :** `src/main/java/fr/tpspringsecurity/config/UserConfig.java`.

Objectif

Remplacer le compte temporaire généré par Spring par deux comptes de TP définis dans l'application, puis stocker leurs mots de passe sous forme BCrypt. Vous configurez **qui peut s'authentifier** et les rôles de ces comptes. Vous ne configurez pas encore **quelles routes chaque rôle peut consulter** : c'est l'exercice 4.

Identifiant exact	Mot de passe de démonstration	Rôle à déclarer
`alice`	`alice123`	`USER`
`bob`	`bob123`	`ADMIN`

Avant de coder

Dans cette branche, Spring Security est déjà installé et sa politique HTTP par défaut est active. Au début, les routes demandent un compte `user` généré. `UserConfig.java` est volontairement incomplet ; ses commentaires indiquent les TODO 3.1 à 3.4. Travaillez uniquement dans ce fichier. Ne créez pas de `SecurityFilterChain` et ne modifiez pas `SecurityConfig.java`.

TODO 3.1 — Déclarer l'encodeur BCrypt

Créez un Bean de type `PasswordEncoder`. Il doit retourner une instance de `BCryptPasswordEncoder`. Cet objet servira à encoder les mots de passe au moment de créer les comptes, puis Spring s'en servira pour vérifier ceux saisis à la connexion.

Imports à repérer : `Bean`, `PasswordEncoder` et `BCryptPasswordEncoder`.

TODO 3.2 — Déclarer la source des utilisateurs

Créez un Bean `UserDetailsService` qui reçoit le `PasswordEncoder` en paramètre :

```
@Bean
public UserDetailsService userDetailsService(PasswordEncoder encoder) {
    // Créer les deux comptes puis les stocker en mémoire.
}
```

Spring injectera le Bean créé au TODO 3.1. La méthode doit fournir au système de sécurité les comptes qu'il utilisera pour authentifier les requêtes.

TODO 3.3 — Construire les deux comptes

Dans cette méthode, construisez un `UserDetails` pour Alice et un pour Bob avec `User.withUsername(...)`. Pour chacun, renseignez l'identifiant du tableau, un mot de passe encodé et le rôle correspondant (`USER` ou `ADMIN`). Terminez chaque construction avec `.build()`.

Attention : passez le mot de passe à `encoder.encode(...)` avant de l'attribuer. Ne stockez pas le texte `alice123` ou `bob123` directement dans l'objet utilisateur. Utilisez `.roles("ADMIN")` pour déclarer le rôle de Bob ; Spring le convertit en autorité `ROLE_ADMIN`.

Imports à repérer : `User`, `UserDetails` et `UserDetailsService`.

TODO 3.4 — Retourner le gestionnaire en mémoire

Retournez un `InMemoryUserDetailsManager` contenant les deux objets construits. Il remplace la source d'utilisateurs automatique pour l'application : après redémarrage, le compte `user` généré n'est plus celui que vous utilisez.

Import à repérer : `InMemoryUserDetailsManager`.

Vérifier dans Planora et avec l'API

Redémarrez Spring. Essayez d'abord l'ancien compte `user`, puis connectez-vous avec Alice et Bob. Pour chacun, consultez **Mes tâches** et **Statistiques**. Essayez enfin Alice avec un mauvais mot de passe. Vous pouvez contrôler une lecture avec curl :

```
curl -i -u alice:alice123 http://localhost:8080/api/tasks
curl -i -u bob:bob123 http://localhost:8080/api/admin/stats
curl -i -u alice:incorrect http://localhost:8080/api/tasks
```

Résultats à observer : Alice et Bob obtiennent `200` sur les tâches et les statistiques ; le mauvais mot de passe reçoit `401`. Les statistiques répondent encore `200` pour Alice : son rôle est bien enregistré, mais aucune règle HTTP ne l'utilise encore. Le chemin `/admin/` ne réserve pas une route à un administrateur à lui seul.

Les mutations peuvent répondre `403` même quand Alice ou Bob est authentifié. C'est la protection CSRF par défaut de cette étape, pas une conséquence de leur rôle. Ne changez pas CSRF ici ; l'exercice 4 configurera les règles HTTP et le comportement des mutations.

TODO 3.5 — Activer le test fourni

Dans `src/test/java/fr/tpspringsecurity/TaskApiSecurityTest.java`, repérez `exercice3AliceEtBobPeuventSeConnec`
Retirez `@Disabled` située juste au-dessus de cette méthode et exécutez :

```
mvn test
```

Ce test vérifie qu'Alice et Bob lisent la liste des tâches (`200`) et qu'un mauvais mot de passe pour Alice est refusé (`401`). Il ne vérifie pas encore que les statistiques sont réservées à Bob : ajoutez cette observation depuis Planora ou curl.

Questions de compréhension

1. Quel rôle joue `PasswordEncoder` à la création du compte puis lors d'une connexion ?
2. Le rôle `ADMIN` protège-t-il déjà `/api/admin/stats` ? Justifiez avec votre observation.
3. Quelle différence faites-vous entre authentifier un compte et autoriser ce compte à utiliser une route ?

Pour continuer : la branche `Tp_version4` contient la correction de `UserConfig.java`. Vous y configurerez les autorisations HTTP.

7. Exercice 4 — Définir les autorisations HTTP (10 min)

Branche : `Tp_version4`. **Fichier à modifier :** `src/main/java/fr/tpspringsecurity/config/SecurityConfig.java`.

Objectif

Les comptes Alice (`USER`) et Bob (`ADMIN`) existent et peuvent tous deux lire les tâches et les statistiques. Vous allez maintenant faire appliquer leurs permissions par Spring Security : laisser une route explicitement publique, limiter les statistiques aux administrateurs, puis exiger une authentification pour le reste de l'API.

Vous allez également configurer HTTP Basic sans session, et adapter CSRF afin que les mutations du client React fourni fonctionnent. Les comptes et leurs rôles restent dans `UserConfig.java`.

Avant de coder

Dans `SecurityConfig.java`, les TODO 4.1 à 4.6 sont des étapes d'une seule chaîne de filtres. Les imports nécessaires ne sont pas tous présents : ajoutez-les depuis les types indiqués ci-dessous. Gardez les règles d'accès dans le même ordre que les étapes.

TODO 4.1 — Déclarer la chaîne de sécurité

Ajoutez l'annotation `@Bean` sur une méthode qui retourne `SecurityFilterChain` et reçoit `HttpSecurity`. La méthode doit configurer puis retourner la chaîne ; c'est là que Spring décide si une requête anonyme ou authentifiée peut atteindre les contrôleurs.

Types à importer : `Bean`, `HttpSecurity`, `SecurityFilterChain`.

TODO 4.2 à 4.4 — Définir et ordonner les règles d'accès

Dans `authorizeHttpRequests`, ajoutez les règles dans cet ordre :

1. Pour `GET /api/public/info`, utilisez `requestMatchers(HttpMethod.GET, "/api/public/info").permitAll()`.
2. Pour `GET /api/admin/**`, utilisez `requestMatchers(HttpMethod.GET, "/api/admin/**").hasRole("ADMIN")`.
3. Pour toutes les requêtes restantes, utilisez `anyRequest().authenticated()`.

Importez `HttpMethod`. Les règles précises doivent être évaluées avant `anyRequest()`, qui termine la liste. `hasRole("ADMIN")` contrôle l'autorité `ROLE_ADMIN`, ajoutée aux comptes avec `.roles("ADMIN")` à l'exercice précédent.

TODO 4.5 — Configurer HTTP Basic et les sessions

Configurez la politique de session avec `SessionCreationPolicy.STATELESS`, puis activez HTTP Basic avec `httpBasic(withDefaults())`. Chaque requête protégée doit alors présenter ses identifiants ; le serveur ne conserve pas une session de connexion entre les appels.

Types à importer : `SessionCreationPolicy` et `withDefaults` de `org.springframework.security.config.Customizer`.

TODO 4.6 — Adapter CSRF au client fourni

Ajoutez `.csrf(csrf -> csrf.disable())` dans la chaîne avant de la construire avec `.build()`.

Ce choix permet au client de TP d'envoyer ses requêtes JSON POST, PUT, PATCH et DELETE avec l'en-tête `Authorization` qu'il construit explicitement, et sans cookie. **Ne généralisez pas cette désactivation à toutes les API stateless ou HTTP Basic** : le navigateur peut envoyer automatiquement des identifiants Basic dans d'autres configurations, ce qui peut exposer à CSRF. Si le mode d'authentification ou le client change, réévaluez la protection. Consultez la documentation Spring Security sur CSRF.

Vérifier les permissions dans Planora

Redémarrez Spring puis vérifiez les cas suivants. Relevez les statuts dans l'onglet Réseau ; les opérations métier doivent réellement réussir ou être refusées par le serveur.

- **Sans connexion :** Mes tâches est refusé avec `401`. La route publique `/api/public/info` doit être vérifiée avec curl et renvoyer `200`.
- **Alice :** elle peut consulter les tâches, créer (`201`), modifier et terminer (`200`), puis supprimer (`204`). L'ouverture des statistiques est refusée avec `403`.
- **Bob :** il peut gérer les tâches et consulter les statistiques (`200`).
- **Projet confidentiel :** dans cette branche, la route n'existe pas encore. Sans compte, Spring refuse d'abord la requête (`401`). Avec Alice ou Bob, la requête passe le filtre HTTP, puis le contrôleur absent entraîne `404`.

```
curl -i http://localhost:8080/api/public/info
curl -i -u alice:alice123 http://localhost:8080/api/admin/stats
curl -i -u bob:bob123 http://localhost:8080/api/admin/stats
```

Activer le test fourni

Dans `TaskApiSecurityTest.java`, retirez `@Disabled` au-dessus de `exercice4LAccesPublicEtLesDroitsAdminSontRespecte` puis lancez `mvn test`. Le test vérifie la route publique, le refus des statistiques à Alice, leur accès par Bob et la création d'une tâche par Alice. Cette dernière requête permet aussi de vérifier que CSRF a bien été configuré pour le client.

Les autres tests actifs vérifient déjà qu'un visiteur ne lit pas les tâches (`401`) et que les deux comptes valides y accèdent (`200`).

Questions de compréhension

1. Pourquoi faut-il placer les règles publiques et ADMIN avant `anyRequest()` ?
2. Pourquoi Alice reçoit-elle `403` sur les statistiques, alors qu'elle est authentifiée ?
3. Que changent `STATELESS` et la désactivation de CSRF pour les requêtes de ce client ? Quelle précaution garder en tête si le client évolue ?

Pour continuer : la branche `Tp_version5` contient la chaîne HTTP complète. Vous y protégerez une méthode Java avec `@PreAuthorize`.

8. Exercice 5 — Protéger une méthode avec `@PreAuthorize` (10 min)

Branche : `Tp_version5`. **Fichiers à modifier :** `src/main/java/fr/tpspringsecurity/controller/TaskController.java` et `src/main/java/fr/tpspringsecurity/config/SecurityConfig.java`.

Objectif

La chaîne HTTP de l'exercice 4 protège déjà les statistiques, mais elle exige seulement une authentification pour les routes restantes sous `/api/tasks`. Vous allez ajouter une deuxième vérification, au niveau d'une méthode Java, pour que seul le rôle `ADMIN` puisse lire le projet confidentiel.

Cet exercice montre la différence entre filtrer une requête HTTP selon son chemin et vérifier les permissions juste avant l'exécution d'une méthode. La ressource métier et sa route sont déjà présentes : il ne faut pas les recréer.

État initial à constater

La branche 5 expose déjà `GET /api/tasks/confidential`. Le contrôleur appelle `taskService.confidential()` et retourne un objet `ConfidentialProject`. À l'ouverture de la branche, la règle HTTP générale demande une authentification, mais Alice peut encore lire la note (`200`). Bob y accède également (`200`) ; un visiteur reçoit `401`.

Connectez-vous comme Alice dans Planora et ouvrez **Projet confidentiel** avant la correction. Relevez le statut. Cette requête est votre état de comparaison.

TODO 5.1 — Protéger la méthode du contrôleur

Dans `TaskController.java`, repérez la méthode existante annotée `@GetMapping("/confidential")` et le commentaire juste au-dessus.

- Importez `org.springframework.security.access.prepost.PreAuthorize` si nécessaire.
- Activez l'annotation `@PreAuthorize("hasRole('ADMIN')")` sur la méthode `confidential()`.
- Laissez son corps appeler le service comme il le fait déjà.

Ne créez pas un nouvel endpoint et ne déplacez pas la logique métier dans le contrôleur.

TODO 5.2 — Activer la sécurité des méthodes

Dans `SecurityConfig.java`, ajoutez `@EnableMethodSecurity` à la classe de configuration. Sans cette annotation, la vérification `@PreAuthorize` n'est pas activée.

N'ajoutez pas de matcher HTTP pour `/api/tasks/confidential`. La règle générale existante exige déjà une identité ; dans cet exercice, la restriction supplémentaire au rôle ADMIN doit venir de la méthode.

Vérifier après redémarrage

Réessayez la route dans Planora avec chaque identité et observez les réponses :

Profil	Résultat attendu
Sans compte	`401` — authentification exigée par la règle HTTP
Alice (`USER`)	`403` — identifiée, mais le rôle ne suffit pas
Bob (`ADMIN`)	`200` — méthode autorisée

`hasRole("ADMIN")` recherche l'autorité `ROLE_ADMIN`, que Spring associe au rôle déclaré avec `.roles("ADMIN")`.

TODO 5.3 — Activer le test fourni

Dans `TaskApiSecurityTest.java`, retirez `@Disabled` juste au-dessus de `exercice5MethodeConfidentielleReserveeAuxAdm` puis lancez `mvn test`. Le test vérifie les trois cas de la matrice (anonyme, Alice, Bob).

Vérifiez aussi que Mes tâches et Statistiques conservent les droits de l'exercice précédent : Alice peut gérer les tâches mais reçoit `403` sur les statistiques ; Bob peut accéder aux deux.

Questions de compréhension

1. Quel est l'état de l'accès d'Alice avant et après l'ajout de `@PreAuthorize` ?
2. Que fait `@EnableMethodSecurity` ?
3. Quelle différence entre la règle générale `authenticated()` et la vérification de rôle sur cette méthode ? Pourquoi les données restent-elles dans le service ?

Pour continuer : la branche `Tp_version6` contient les deux annotations actives et les tests complets.

9. Exercice 6 — Vérifier le parcours final (5 min)

Branche : `Tp_version6`. La correction complète est fournie ; aucun TODO de sécurité à compléter.

```
mvn clean verify
```

Le test d'intégration couvre l'accès public, les identifiants invalides, les accès aux statistiques et à la note, les mutations d'Alice, l'évolution des statistiques et la suppression d'une tâche inexistante. Les tests unitaires vérifient la logique métier. Tous les tests sont actifs en version 6.

La matrice suivante suppose des données valides et un identifiant de tâche existant :

Opération	Anonyme	Alice `USER`	Bob `ADMIN`
`GET /api/public/info`	200	200	200
`GET /api/tasks`	401	200	200
`POST /api/tasks`	401	201	201
`PUT /api/tasks/{id}`	401	200	200
`PATCH /api/tasks/{id}/complete`	401	200	200
`DELETE /api/tasks/{id}`	401	204	204
`GET /api/admin/stats`	401	403	200
`GET /api/tasks/confidential`	401	403	200

Dans Planora, refaites un cycle complet de gestion des tâches avec Alice, puis vérifiez les statistiques et le projet confidentiel avec Bob. Recommencez ces accès avec Alice et sans compte. Utilisez curl ou l'onglet Réseau pour contrôler les statuts précis.

10. Aide et bilan

Symptôme	Vérification
API inaccessible	Spring doit écouter sur le port 8080
`401`	Identifiants absents ou invalides ; vérifier la branche et le compte disponible
Mutation `403` en étapes 2 et 3	CSRF par défaut ; comportement attendu sans jeton
Mutation `403` après l'exercice 4	Vérifier le TODO 4.6 et le redémarrage de Spring
Statistiques `403` avec Alice après l'exercice 4	Comportement attendu : rôle `ADMIN` requis
Note `404` en branches 1 à 4 avec un compte autorisé	Route absente, fournie seulement à partir de la branche 5
Alice lit la note après l'exercice 5	Vérifier `@PreAuthorize` et `@EnableMethodSecurity`
Tests ignorés	Retirer `@Disabled` seulement après avoir terminé l'exercice correspondant
Ancien comportement après un changement de branche	Arrêter puis relancer Spring et Vite

`401` indique une authentification absente ou invalide. `403` indique un refus de sécurité : rôle insuffisant dans le parcours final, ou jeton CSRF absent aux étapes précédentes. `404` indique une route ou une tâche absente, lorsque la sécurité autorise l'appel.

Bilan à rendre : fiche d'analyse, configurations des exercices, résultats des tests et observations dans Planora. Expliquez la distinction entre authentification et autorisation, l'intérêt de BCrypt, l'ordre des règles HTTP et l'activation de la sécurité des méthodes.

HTTP Basic encode les identifiants en Base64 ; il ne les chiffre pas. Les comptes et données de ce TP sont fictifs et destinés au local. Cette démonstration n'est pas une configuration de production.

Ressources : `ENONCE.md` pour l'exercice courant ; `docs/corrections/` pour les corrigés précédents ; `docs/ARCHITECTURE.md`, `docs/SCENARIOS_DE_TEST.md` et `docs/GUIDE_HTTP.md` pour les détails.